ДИСЦИПЛИНА	Технологии индустриального программирования
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Адышкин Сергей Сергеевич
СЕМЕСТР	1 семестр, 2025-2026 гг.

Практическое занятие №3.

Реализация простого HTTP-сервера на стандартной библиотеке net/http. Обработка запросов GET/POST

Цели занятия

- Освоить базовую работу со стандартной библиотекой `net/http` без сторонних фреймворков.
- Научиться поднимать HTTP-сервер, настраивать маршрутизацию через `http.ServeMux`.
- Научиться обрабатывать параметры запроса (query, path), тело запроса (JSON/form-data) и формировать корректные ответы (код статуса, заголовки, JSON).
- Научиться базовому логированию запросов и обработке ошибок.

Планируемые результаты

1. Запускать простой HTTP-сервер и настраивать маршруты.
2. Реализовывать обработчики (`http.HandlerFunc`) для GET/POST.
3. Читать query-параметры, path-параметры (простым способом), тело запроса, декодировать JSON.
4. Возвращать JSON-ответы с корректными кодами статуса и заголовками.
5. Писать простую прослойку логирования (middleware-подход на `http.Handler`).
6. Тестировать API с помощью `curl`/PowerShell/HTTPie/Postman.

Предварительные требования

- Установлен Go (1.21+).
- Базовые знания Go: функции, структуры, модули.

- Опыт работы с ПЗ №1–2 (go mod, структура проекта, запуск).

Материалы и ПО

- IDE: GoLand / VS Code (Go extension) / любая.
- Инструменты для тестирования API: `curl` (Windows PowerShell, macOS, Linux), Postman/Insomnia (по желанию).

Теоретическая часть

Архитектура `net/http`

- `http.ListenAndServe(addr string, handler http.Handler)` — запуск сервера.
- `http.Handler`, `http.HandlerFunc` — контракт обработки запроса.
- `http.Request` — всё о запросе: метод, URL, заголовки, тело (`Body`).
- `http.ResponseWriter` — то, куда пишем ответ, статус и заголовки.
- `http.ServeMux` — простой роутер из стандартной библиотеки.

Обработка запросов

- **GET**: данные в query (`/items?limit=10`), иногда — в пути (`/items/123`).
- **POST**: данные в теле — чаще JSON или `application/x-www-form-urlencoded`/`multipart/form-data`.
- Кодировки/заголовки: `Content-Type`, `Accept`, `Authorization` и т.д.
- Коды ответа: 200/201/204, 400/404/422, 500.

Мини-middleware

- Оборачиваем `http.Handler` собственной функцией, добавляющей логирование времени, метода, пути, кода ответа.

Middleware — это промежуточный слой (прослойка) между сервером и конечным обработчиком запроса. В Go (и в backend-разработке в целом) middleware выполняет «сквозные» задачи, которые должны работать для

многих или всех маршрутов приложения: логирование, аутентификация, авторизация, обработка ошибок, CORS, ограничение частоты запросов и т. д.

По сути, middleware перехватывает входящий HTTP-запрос до того, как он попадёт в основной обработчик, и может:

- изменить запрос (например, добавить данные в контекст);
- проверить его (например, убедиться, что токен валиден);
- прервать выполнение (вернуть 401/403 и не пускать дальше);
- или пропустить его к следующему обработчику.

Пример:

```
    // Простейший middleware для логирования
    func LoggingMiddleware(next http.Handler) http.Handler {
        return  http.HandlerFunc(func(w  http.ResponseWriter,  r
*http.Request) {
            log.Printf("%s %s", r.Method, r.URL.Path)
            next.ServeHTTP(w, r) // передаём управление дальше
        })
    }
```

Преимущества подхода:

- **Единая точка контроля** — всё, что касается авторизации, логов или CORS, вынесено в одно место.
- **Переиспользование** — middleware можно подключать к любым маршрутам.
- **Композиция** — middleware образуют цепочку: Recovery → Logging → CORS → Auth → Handler.

Таким образом, middleware в Go — это удобный способ реализовать повторяющиеся задачи обработки запросов без дублирования кода в каждом хендлере.

Практическая часть

Подготовка проекта

Структура (минимально):

```
pz3-http/
  cmd/
    server/
      main.go
  internal/
    api/
      handlers.go
      middleware.go
      responses.go
    storage/
      memory.go
  go.mod
```

Инициализация:

```
# Windows PowerShell / macOS / Linux
mkdir -p pz3-http/cmd/server pz3-http/internal/api pz3-http/internal/storage
cd pz3-http
go mod init example.com/pz3-http
```

1) Первый сервер и маршрут GET `/health`

cmd/server/main.go:

```
package main

import (
  "log"
  "net/http"
)

func main() {
  mux := http.NewServeMux()
  mux.HandleFunc("GET /health", func(w http.ResponseWriter, r *http.Request) {
```

```
        w.Header().Set("Content-Type", "application/json")
        w.WriteHeader(http.StatusOK)
        w.Write([]byte(`{"status":"ok"}`))
    })

    addr := ":8080"
    log.Println("listening on", addr)
    if err := http.ListenAndServe(addr, mux); err != nil {
        log.Fatal(err)
    }
}
```

Проверка:

```
# macOS/Linux
curl -i http://localhost:8080/health
# Windows PowerShell
curl http://localhost:8080/health
```

Ожидаем `200 OK` и JSON `{"status":"ok"}`.

2) Память в приложении и модель сущности

Создадим простую сущность `Task` и in-memory хранилище.

`internal/storage/memory.go`:

```
package storage

import (
    "errors"
    "sync"
)

type Task struct {
    ID    int64  `json:"id"`
    Title string `json:"title"`
    Done  bool   `json:"done"`
}

type MemoryStore struct {
```

```
 mu    sync.RWMutex
 auto  int64
 tasks map[int64]*Task
}

func NewMemoryStore() *MemoryStore {
 return &MemoryStore{
       tasks: make(map[int64]*Task),
 }
}

func (s *MemoryStore) Create(title string) *Task {
 s.mu.Lock()
 defer s.mu.Unlock()
 s.auto++
 t := &Task{ID: s.auto, Title: title, Done: false}
 s.tasks[t.ID] = t
 return t
}

func (s *MemoryStore) Get(id int64) (*Task, error) {
 s.mu.RLock()
 defer s.mu.RUnlock()
 t, ok := s.tasks[id]
 if !ok {
       return nil, errors.New("not found")
 }
 return t, nil
}

func (s *MemoryStore) List() []*Task {
 s.mu.RLock()
 defer s.mu.RUnlock()
 out := make([]*Task, 0, len(s.tasks))
 for _, t := range s.tasks {
       out = append(out, t)
```

```
  }
  return out
}
```

3) Вспомогательные ответы и ошибки

`internal/api/responses.go`:

```
package api

import (
  "encoding/json"
  "net/http"
)

type ErrorResponse struct {
  Error string `json:"error"`
}

func JSON(w http.ResponseWriter, status int, v any) {
  w.Header().Set("Content-Type", "application/json")
  w.WriteHeader(status)
  _ = json.NewEncoder(w).Encode(v)
}

func BadRequest(w http.ResponseWriter, msg string) {
  JSON(w, http.StatusBadRequest, ErrorResponse{Error: msg})
}

func NotFound(w http.ResponseWriter, msg string) {
  JSON(w, http.StatusNotFound, ErrorResponse{Error: msg})
}

func Internal(w http.ResponseWriter, msg string) {
  JSON(w,                    http.StatusInternalServerError,
ErrorResponse{Error: msg})
}
```

4) Обработчики: GET `/tasks` и POST `/tasks`

internal/api/handlers.go:

```
package api

import (
 "encoding/json"
 "errors"
 "net/http"
 "strconv"
 "strings"

 "example.com/pz3-http/internal/storage"
)

type Handlers struct {
 Store *storage.MemoryStore
}

func NewHandlers(store *storage.MemoryStore) *Handlers {
 return &Handlers{Store: store}
}

// GET /tasks
func (h *Handlers) ListTasks(w http.ResponseWriter, r
*http.Request) {
 tasks := h.Store.List()

 // Поддержка простых фильтров через query: ?q=text
 q := strings.TrimSpace(r.URL.Query().Get("q"))
 if q != "" {
       filtered := tasks[:0]
       for _, t := range tasks {
             if strings.Contains(strings.ToLower(t.Title),
strings.ToLower(q)) {
                   filtered = append(filtered, t)
             }
```

```
            }
            tasks = filtered
        }

        JSON(w, http.StatusOK, tasks)
    }

    type createTaskRequest struct {
        Title string `json:"title"`
    }

    // POST /tasks
    func (h *Handlers) CreateTask(w http.ResponseWriter, r
*http.Request) {
        if r.Header.Get("Content-Type") != ""
&& !strings.Contains(r.Header.Get("Content-Type"),
"application/json") {
            BadRequest(w, "Content-Type must be
application/json")
            return
        }

        var req createTaskRequest
        if err := json.NewDecoder(r.Body).Decode(&req); err != nil
{
            BadRequest(w, "invalid json: "+err.Error())
            return
        }
        req.Title = strings.TrimSpace(req.Title)
        if req.Title == "" {
            BadRequest(w, "title is required")
            return
        }

        t := h.Store.Create(req.Title)
        JSON(w, http.StatusCreated, t)
```

```
    }

    // GET /tasks/{id} (простой path-парсер без стороннего
роутера)
    func (h *Handlers) GetTask(w http.ResponseWriter, r
*http.Request) {
     // Ожидаем путь вида /tasks/123
     parts := strings.Split(strings.Trim(r.URL.Path, "/"), "/")
     if len(parts) != 2 {
           NotFound(w, "invalid path")
           return
     }
     id, err := strconv.ParseInt(parts[1], 10, 64)
     if err != nil {
           BadRequest(w, "invalid id")
           return
     }

     t, err := h.Store.Get(id)
     if err != nil {
           if errors.Is(err, errors.New("not found")) {
                 NotFound(w, "task not found")
                 return
           }
           Internal(w, "unexpected error")
           return
     }
     JSON(w, http.StatusOK, t)
    }
```

5) Простое логирование (middleware-обёртка)

`internal/api/middleware.go`:

```
    package api

    import (
     "log"
```

```
    "net/http"
    "time"
)

type statusRecorder struct {
    http.ResponseWriter
    status int
}

func (sr *statusRecorder) WriteHeader(code int) {
    sr.status = code
    sr.ResponseWriter.WriteHeader(code)
}

func Logging(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
*http.Request) {
        start := time.Now()
        rec := &statusRecorder{ResponseWriter: w, status:
200}
        next.ServeHTTP(rec, r)
        log.Printf("%s %s %d %v", r.Method, r.URL.Path,
rec.status, time.Since(start))
    })
}
```

6) Сборка маршрутов в `main.go`

Обновим `cmd/server/main.go`:

```
package main

import (
    "log"
    "net/http"

    "example.com/pz3-http/internal/api"
    "example.com/pz3-http/internal/storage"
```

```
)

func main() {
  store := storage.NewMemoryStore()
  h := api.NewHandlers(store)

  mux := http.NewServeMux()
  mux.HandleFunc("GET /health", func(w http.ResponseWriter,
r *http.Request) {
        api.JSON(w, http.StatusOK,
map[string]string{"status": "ok"})
  })

  // Коллекция
  mux.HandleFunc("GET /tasks", h.ListTasks)
  mux.HandleFunc("POST /tasks", h.CreateTask)
  // Элемент
  mux.HandleFunc("GET /tasks/", h.GetTask)

  // Подключаем логирование
  handler := api.Logging(mux)

  addr := ":8080"
  log.Println("listening on", addr)
  if err := http.ListenAndServe(addr, handler); err != nil {
        log.Fatal(err)
  }
}
```

7) Тестирование API

GET /health

```
curl -i http://localhost:8080/health
```

POST /tasks (создание)

macOS/Linux:

```
curl -i -X POST http://localhost:8080/tasks \
  -H "Content-Type: application/json" \
```

```
  -d '{"title":"Купить молоко"}'
```

Windows PowerShell:

```
curl -Method POST http://localhost:8080/tasks `
  -Headers @{"Content-Type"="application/json"} `
  -Body '{"title":"Купить молоко"}'
```

Ожидаем `201 Created` и JSON с `id`.

GET /tasks (список)

```
curl -i http://localhost:8080/tasks
curl -i "http://localhost:8080/tasks?q=молоко"
```

GET /tasks/{id}

```
curl -i http://localhost:8080/tasks/1
```

Ошибки

- `POST /tasks` без `title` → `400 Bad Request`
- `GET /tasks/abc` → `400 Bad Request`
- `GET /tasks/9999` → `404 Not Found`

8) Дополнительно (по желанию/звёздочка)

1. **CORS (минимально):** добавить заголовки `Access-Control-Allow-Origin: *` для GET/POST (в отдельной middleware).
2. **Валидация длины `title`** (например, 1…140 символов).
3. **Метод PATCH /tasks/{id}** для отметки `Done=true`.
4. **Метод DELETE /tasks/{id}`**.
5. **Graceful shutdown** через `http.Server` и контекст.
6. **Юнит-тесты обработчиков** с `httptest`.

Пояснения под Windows / macOS / Linux

- **Запуск:**

```
cd pz3-http
go run ./cmd/server
```

- **Сборка бинарника:**

```
# Windows (из PowerShell):
go build -o .\bin\server.exe .\cmd\server
.\bin\server.exe

# macOS/Linux:
go build -o ./bin/server ./cmd/server
./bin/server
```

- Порт 8080 часто свободен. Если занят — смените `addr := ":9090"`.

Мини-шпаргалка (раздатка)

Коды статуса:

- 200 OK — успешный ответ
- 201 Created — ресурс создан
- 204 No Content — успешно, без тела
- 400 Bad Request — неверные данные
- 404 Not Found — ресурс не найден
- 422 Unprocessable Entity — некорректные данные по смыслу
- 500 Internal Server Error — внутренняя ошибка

Частые заголовки:

- `Content-Type: application/json` — тип тела ответа/запроса
- `Accept: application/json` — клиент ожидает JSON
- `Authorization: Bearer <token>` — авторизация (позже)
- `Access-Control-Allow-Origin: *` — упрощённый CORS (для фронта)

Шаблоны обработчиков:

```
func(w http.ResponseWriter, r *http.Request) {
  // читать query
  r.URL.Query().Get("q")
```

```
    // читать заголовки
    r.Header.Get("X-My-Header")
    // читать тело (JSON)
    json.NewDecoder(r.Body).Decode(&payload)
    // писать ответ
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusOK)
    json.NewEncoder(w).Encode(data)
}
```

Подготовка отчёта

Обязательная часть (в аудитории)

1. Поднимите сервер на `:8080` и реализуйте маршруты:
   - `GET /health` — возвращает `{"status":"ok"}`.
   - `GET /tasks` — возвращает список задач, поддерживает фильтр `?q=`.
   - `POST /tasks` — создаёт задачу по `{"title":"..."}`.
   - `GET /tasks/{id}` — возвращает одну задачу.
2. Реализуйте единый helper для JSON-ответов и ошибок (как в примере).
3. Добавьте middleware-логирование (метод, путь, статус, время).
4. Напишите 6–8 тестовых запросов `curl` (или коллекцию Postman) и сохраните их в файле `requests.md`.

Дополнительная часть (на выбор)

- `PATCH /tasks/{id}` для изменения флага `done`.
- `DELETE /tasks/{id}`.
- Валидация: `title` не короче 3 и не длиннее 140 символов — иначе 422.
- Простейший CORS-middleware.

Домашнее задание

- Вынесите конфигурацию порта в переменную окружения `PORT` (по умолчанию `8080`).
- Добавьте `Makefile` или `PowerShell`-скрипт с целями `run`, `build`, `test`.
- Покройте 1–2 обработчика юнит-тестами (`httptest.NewRecorder`).

Критерии оценивания (10 баллов)

1. Сервер запускается, маршруты работают — **2 б.**

2. Корректные коды ответа, заголовки, JSON — **2 б.**
3. Обработка ошибок и валидация входных данных — **2 б.**
4. Middleware-логирование — **2 б.**
5. Набор тестовых запросов (`requests.md`) + понятная структура кода — **2 б.**

(+ до 3 доп. баллов за “звёздочку”: PATCH/DELETE, CORS, тесты, graceful shutdown.)

Частые ошибки и как их избежать

- **Забыли `Content-Type`** при ответе JSON → клиенты не парсят тело как JSON.
- **Неверный `Content-Type` на POST** → проверяйте `application/json`.
- **Игнорирование ошибок `json.Decoder`** → всегда проверяйте `err`.
- **Паника при пустом теле** → обрабатывайте `nil`/пустые поля, закрывайте `Body` при необходимости.
- **Жёсткий парсинг path** → аккуратно разбирайте `r.URL.Path`, проверяйте длины массивов.